

DCCS307 · MODULE 5 · CASE STUDY PROJECT

CN_Module5_Group7

Topic · Network Latency in High-Frequency Trading

A protocol-level study of TCP vs UDP, kernel bypass, and how real exchanges trade microseconds for millions.

Boseok Kim
PROBLEM · LATENCY ANALYSIS

Jungha Kim
KERNEL BYPASS SOLUTIONS

Hanju Son
EMPIRICAL BENCHMARK

Junseo Lee
INDUSTRY APPLICATIONS

2026.05 · COMPUTER NETWORKS · GROUP 07

Four Parts, **One Question**: where do the microseconds go?

PART 01

The Problem

What is HFT, why microseconds matter, and which of the four network delays we can actually control.

Boseok Kim

PART 02

The Solution

Kernel bypass — how DPDK, OpenOnload, and RDMA attack the d_proc bottleneck.

Jungha Kim

PART 03

Our Experiment

TCP vs UDP measured end-to-end with Docker + tc netem. Our primary data.

Hanju Son

PART 04

The Industry

NASDAQ, CME, NYSE, Citadel — how the real world applies all of the above.

Junseo Lee

Problem → Solution → Validation → Application

— 01 · THE PROBLEM

What is HFT — and why does $1\mu\text{s}$ matter?

High-Frequency Trading is algorithmic, microsecond-scale automated trading. Orders are decided and submitted by machines, not humans.

- ▶ ~50% of all US equity trading volume is HFT.
- ▶ A 1 ms latency advantage is worth roughly **tens of millions / year** to a major bank.
- ▶ Strategies live and die on *who arrives first*, not who is cleverest.

This is not a niche problem. It's a 50%-of-the-market problem.

~50%
of US equity volume
DRIVEN BY HFT

Tens of \$M
per year, per ms
INDUSTRY ESTIMATES

01 · THE PROBLEM

Winner-Takes-All — tail latency beats mean

When a price moves, only **the first arriving order** takes the trade. Second place wins nothing.

- ▶ A system with mean **100 μ s** but P99 of **200 ms** is effectively unusable.
- ▶ The mean lies. The tail is where the money lives — or evaporates.
- ▶ Our target metric for the rest of the talk: **P99 RTT under packet loss**.

HFT lives or dies at the tail, not at the average.



Latency is not Gaussian. The right tail is exactly the regime HFT cares about.

The Four Sources of Network Delay

From Kurose & Ross — every packet pays four taxes on the way to its destination:

d_{proc} Processing Header inspection, kernel TCP/IP, syscall & copy overhead.	d_{queue} Queuing Time waiting in router / NIC buffers before transmission.	d_{trans} Transmission L / R — bits pushed onto the link at line rate.	d_{prop} Propagation d / s — signal traveling at near light-speed through fiber.
--	---	---	---

$d_{\text{total}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$ — this slide is the foundation for everything that follows.

Reference: Kurose & Ross, Computer Networking — A Top-Down Approach, Chapter 1.

Which of these can software **actually** move?

DELAY	CONTROL LEVER	DOMAIN	PROJECT RELEVANCE
d_{prop}	Co-location, microwave towers, hollow-core fiber	Real estate / physics	Fixed constant
d_{trans}	10 / 40 / 100 GbE link upgrade	Infrastructure	Effectively constant
d_{proc}	Kernel bypass — DPDK, OpenOnload, RDMA	Software	Part 2 (Jungha)
d_{queue}	Protocol choice, QoS, buffer sizing	Software	Part 3 (Hanju)

Light speed is fixed. Software isn't. We can only optimize the two software-controlled delays — and they happen to be the two that dominate microsecond-scale systems.

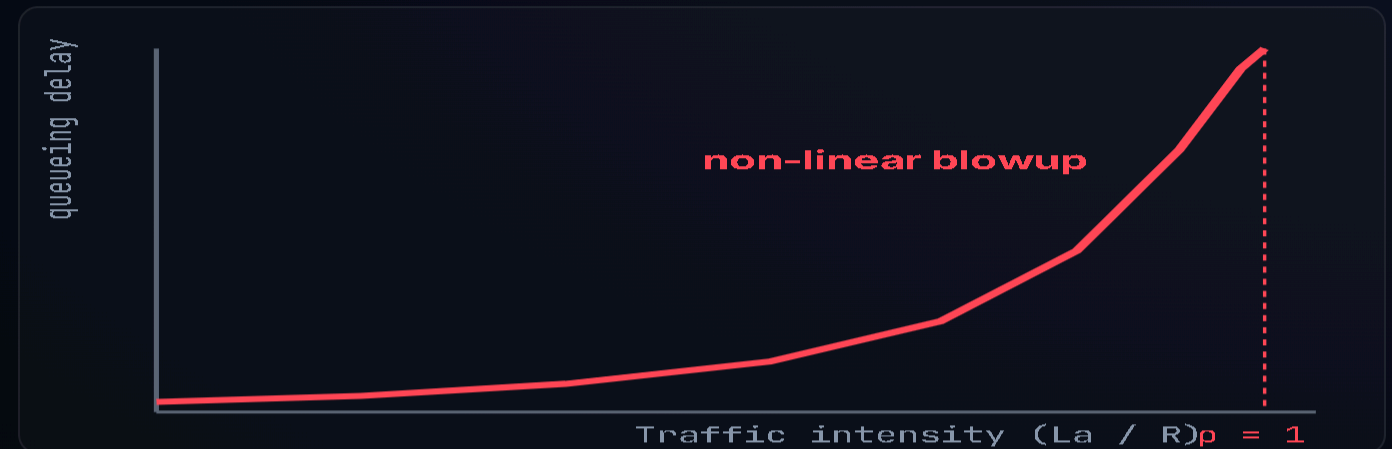
01 · THE PROBLEM

d_{queue} is **non-linear** — that's the real danger

As traffic intensity $\rho = \lambda a / R \rightarrow 1$, queuing delay does not grow linearly — it **explodes**.

- ▶ Average delay is fine until utilization approaches capacity.
- ▶ Then a single burst of loss / re-tx forces buffers to back up — and tail latency goes from microseconds to **milliseconds or worse**.
- ▶ **Flash Crash (2014)**: 10-year US Treasury fell 33–34 bp, then recovered, in ~10 minutes — a real-world demonstration of d_{queue} non-linearity.

Average is fine until $\rho \rightarrow 1$. Then everything explodes.



Kurose & Ross — packet queueing delay vs traffic intensity.

Next → So how does the industry actually escape this kernel-level bottleneck? **Jungha** will show you.

1 μ s = Millions — **attack d_{proc} first**

Boseok identified d_{proc} as one of the two software-controlled delays. In a standard TCP/IP stack, almost all of d_{proc} is spent *inside the Linux kernel*.

- ▶ A single round-trip enters the kernel **twice** (RX + TX).
- ▶ Each entry is 10–30 μ s — **20–60 μ s wasted per round-trip**.
- ▶ Top firms target **tick-to-trade < 10 μ s** — impossible if the kernel is on the path.

The kernel is not slow because it's badly written. It's slow *by design* — for safety, fairness, and generality. HFT needs none of those.

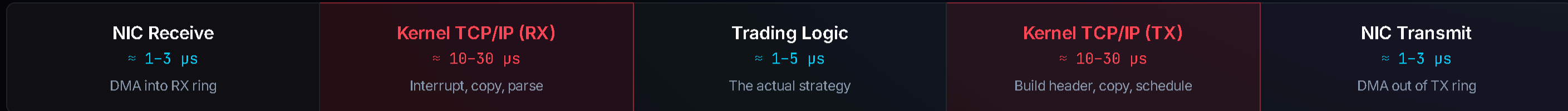
20–60

μ s wasted in the kernel — per RTT

RX ENTRY + TX ENTRY

The Kernel **is** the Bottleneck

A single trading round-trip, dissected — the red blocks are pure overhead:



The strategy itself takes 1–5 μ s. The kernel takes **20–60 μ s**. We are spending roughly 10× more time shuffling packets than thinking about trades.

Source: industry whitepapers (Solarflare, Intel DPDK Programmer's Guide). Numbers depend on NIC, CPU, and tuning.

Why the Kernel Wastes **Microseconds**

① Interrupts

Every incoming packet raises an IRQ. Context switch + cache pollution → cost per packet, not per batch.

② Memory copies

Packet copied from NIC ring → kernel `sk_buff` → user buffer. Each copy is hundreds of nanoseconds at best.

③ Context switching

System calls (`recv` / `send`) cross the user/kernel boundary. Each crossing flushes registers and TLB.

④ Protocol processing

TCP state machine, congestion control, checksum, scheduling — written for generality, not for 1 μ s targets.

If the kernel is the bottleneck, the answer is to skip it entirely. That is exactly what kernel-bypass technologies do.

Three Bypass Approaches — Performance vs Compatibility

OpenOnload

≈ 1–3 μs

User-space TCP/IP stack injected via LD_PRELOAD.
No code change. Requires Solarflare NIC.

Best for: drop-in latency win.

DPDK

< 1 μs

Poll-mode driver. App owns the NIC directly. **Full rewrite required.** Huge pages, NUMA pinning, busy-poll cores.

Best for: greenfield ultra-low latency.

RDMA

≈ 1–2 μs

NIC writes directly to remote memory. Remote CPU not involved. Requires **InfiniBand** or **RoCE** fabric.

Best for: intra-datacenter.

The trade is always the same: the more of the kernel you bypass, the faster you go — and the more your code and hardware must change.

Deep Dive — How each one skips the kernel

DPDK	Poll-mode driver — no interrupts. App busy-polls the NIC ring directly. Huge pages avoid TLB misses. NUMA pinning keeps memory local.	< 1 μ s
OPENONLOAD	User-space TCP/IP linked via LD_PRELOAD — existing <code>recv()</code> / <code>send()</code> calls are rerouted. Same socket API. Zero code change.	1–3 μ s
RDMA	NIC reads/writes remote process memory directly. Remote CPU bypassed entirely. Uses verbs API (queue pairs, work requests) — not BSD sockets.	1–2 μ s
STANDARD TCP	For comparison — kernel handles every packet. Interrupts, copies, syscalls, protocol state. Compatible with everything; fast at nothing.	30–60 μ s

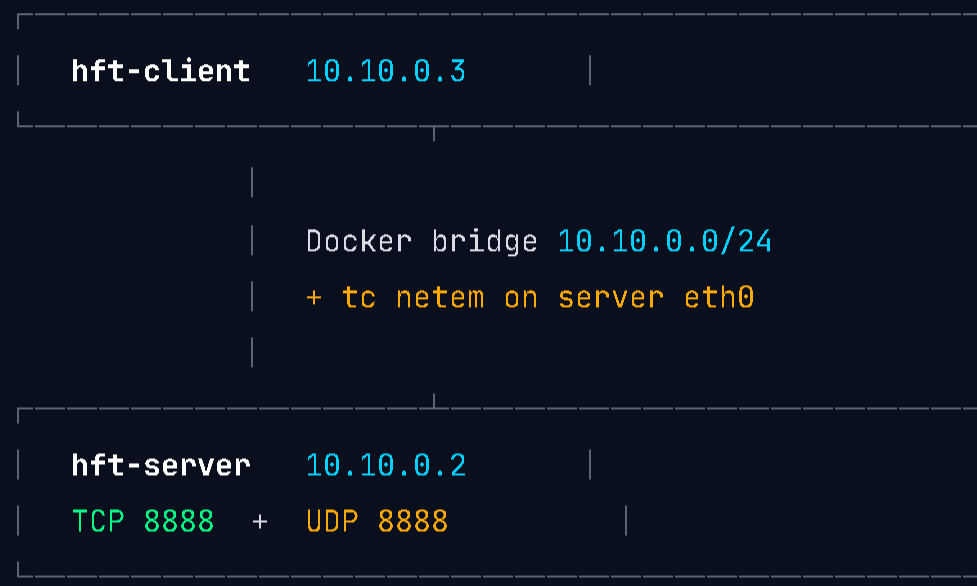
Bottom Line: the comparative matrix

OPTION	LATENCY	CODE CHANGE	HARDWARE	BEST FOR
Standard TCP/IP	30-60 μ s · HOL $\rightarrow$ 200 ms+ RTT on loss	None	Any NIC	Compatibility
OpenOnload	1-3 μ s	None (LD_PRELOAD)	Solarflare NIC	Drop-in kernel bypass
DPDK	< 1 μ s	Full rewrite	DPDK-capable NIC	Greenfield ultra-low
RDMA	1-2 μ s · zero-copy	New verbs API	InfiniBand / RoCE	Intra-datacenter

The more kernel you bypass, the faster you go — and the more your code and hardware must change.

Next $\rightarrow$ These numbers are from industry whitepapers. But what about the underlying TCP vs UDP gap? Can we measure it ourselves? Hanju ran the experiment.

Our Setup — Docker + tc netem



A real NIC path — not loopback. tc netem on the server's egress gives us reproducible loss, delay, and jitter.

Why Docker, not localhost?

Loopback bypasses the NIC. We need a **real interface** to observe protocol-level behavior.

Why tc netem?

Reproducible packet **loss / delay / jitter** injected at the kernel queueing discipline layer.

What we built

Python 3 + raw sockets + Docker. Pipe-delimited order protocol. 10,000 orders / run, 100 μ s spacing.

We built our own HFT simulator end-to-end. This is our team's primary data.

Baseline — TCP ≈ UDP (no loss)

METRIC	TCP	UDP	GAP
Mean RTT	188.6 μs	178.0 μs	+10.6 μs
P99 RTT	375.7 μs	386.1 μs	≈ 0
Delivered	1000 / 1000	1000 / 1000	—

Without packet loss, the choice barely matters. Both protocols look nearly identical — a ~10 μs gap on the mean.

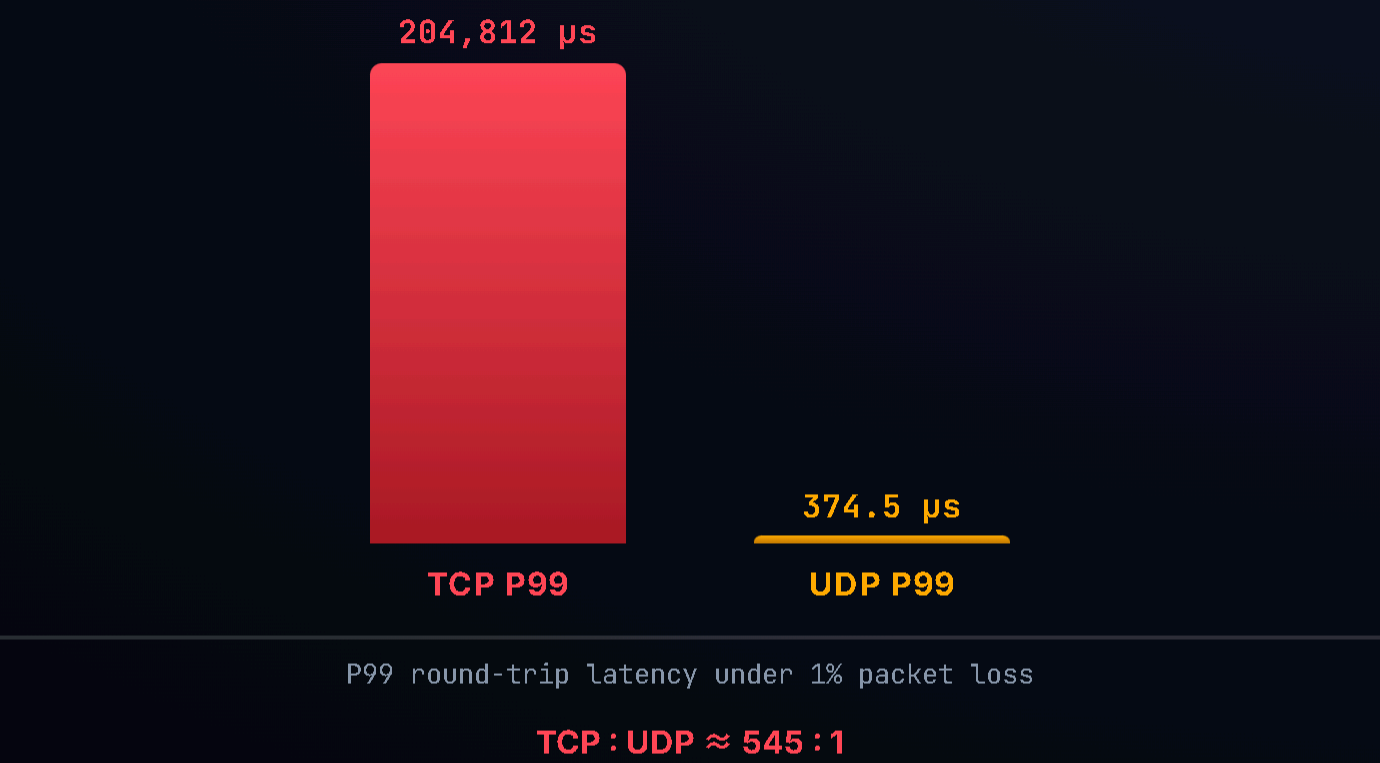


A naive reading would conclude "protocol choice doesn't matter for HFT". Let's see what happens when we add 1% packet loss.

The Bombshell — 1% packet loss changes everything

METRIC	TCP	UDP
Mean RTT	2,306 μ s	177.9 μ s
P99 RTT	204,812 μ s	374.5 μ s
P99 (human)	$\approx$ 205 ms	$\approx$ 0.37 ms
Delivered	1000 / 1000	992 / 1000

TCP's P99 exploded by ~545 $\times$ from a 1% loss rate. UDP's tail barely moved.



Why? Linux RTO_min = 200 ms

TCP cannot detect a lost packet faster than its **Retransmission Timeout**. On Linux, the floor of that timeout is hardcoded:

```
// net/ipv4/tcp.c (Linux kernel)
#define TCP_RTO_MIN    ((unsigned)(HZ/5))
                        // = 200 ms
```

On a clean network, RTT is $\sim 180 \mu\text{s}$. But if a packet is lost, TCP *waits at least 200 ms* before even retrying.

TCP guarantees delivery — but its guarantee mechanism is the latency.

The mechanism, step by step

1. Order packet sent.
2. **Lost** by tc netem.
3. Receiver never sees it $\rightarrow$ no ACK.
4. Sender's RTO timer expires after $\geq 200 \text{ ms}$.
5. Retransmit. ACK finally arrives.
6. RTT for that one order: **$\sim 205 \text{ ms}$** .

This is exactly Boseok's **d_{queue} non-linear blowup** — measured on our own machine.

UDP Trade-off — 8/1000 lost, but the tail stays flat

- ▶ Under 1% loss, UDP loses **8 out of 1000** orders.
- ▶ The other **992** arrive with the same latency as the no-loss baseline.
- ▶ UDP doesn't try to recover. **That refusal is its speed.**

UDP refuses to wait. That refusal is its speed.

FOR HFT...	LOSS	205 MS TAIL
Market data feed (price ticks)	Next tick fixes it	Trade opportunity gone
Order entry (sending a trade)	Lost order = lost trade	Tolerable (rarely happens)

The asymmetry, in plain terms

For a **market data feed**, missing one price tick is recoverable — the next update replaces it in microseconds.

For the *same feed*, a 200 ms TCP stall is **permanent** — during those 200 ms, the price has moved 1000+ times, and the trading opportunity is gone.

Different workloads, different protocols. UDP for speed-critical broadcasts. TCP for correctness-critical orders. The industry knows this — we'll see exactly how in Part 4.

What Our Data Proves

01

Boseok's theory, measured

"d_{queue} is non-linear" — confirmed. A 1% loss rate produced a **~545×** blowup in P99 latency. ✓

02

Jungha's numbers, recoverable

Standard TCP/IP at 30–60 μs (whitepaper) vs our 188 μs baseline — the gap is Python interpreter overhead. **Consistent** ✓

03

TCP guarantees ≠ TCP fits

TCP's reliability is real but its *mechanism* (RTO ≥ 200 ms) is incompatible with sub-millisecond markets.

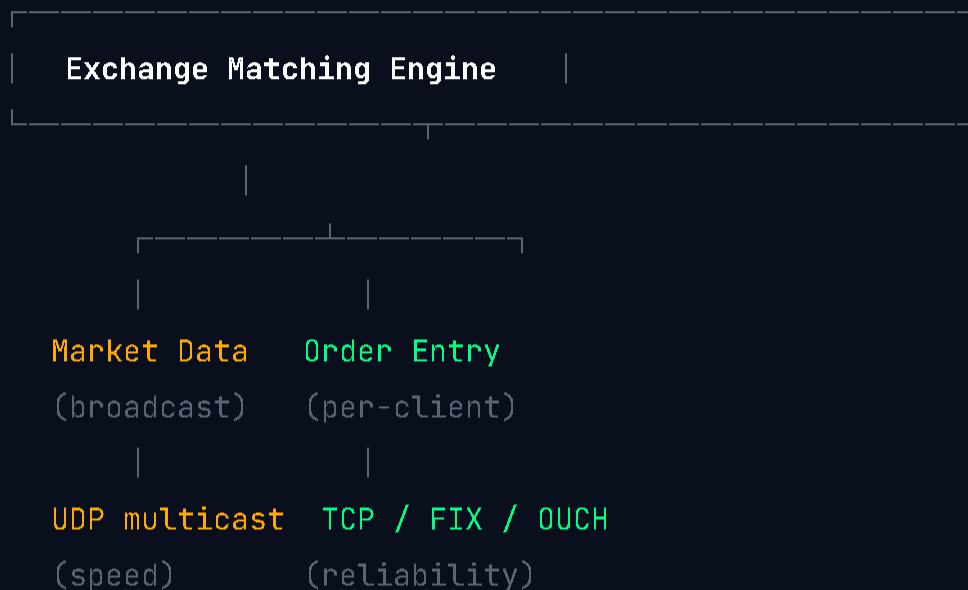
04

UDP wins where loss is recoverable

8 lost out of 1000 — but P99 stayed at 374 μs. **Tail held flat under exactly the stress that broke TCP.**

Next → Our numbers explain why real exchanges split their protocols. **Junseo** will show you who, where, and how.

The Industry's Answer — split the protocol



The matching engine doesn't pick a single protocol. It picks two, by purpose.

UDP Multicast — Market Data

Same prices broadcast to every subscriber simultaneously. Loss is tolerable; **head-of-line blocking is not**.

TCP / FIX / OUCH — Order Entry

One order, sent once, must arrive. Reliability matters more than tail latency — and TCP's tail is rare when there's no broadcast contention.

Real exchanges don't pick TCP or UDP. They use both, by purpose.

Three Exchanges — same pattern

EXCHANGE	MARKET DATA	ORDER ENTRY
NASDAQ	TotalView-ITCH over Mo1dUDP64 — UDP multicast	OUCH over TCP
CME (Chicago Mercantile)	MDP 3.0 — UDP multicast + SBE binary encoding	iLink 3 over TCP
NYSE	Pillar — UDP multicast, redundant A/B lines	Pillar FIX Gateway over TCP

Every major exchange uses the same split: UDP multicast for the broadcast, TCP for the per-client transactional path. Independently invented, independently the same answer.

Sources: NASDAQ TotalView-ITCH spec; CME MDP 3.0 documentation; NYSE Pillar technical docs.

The Clever Hybrid — CME's UDP + TCP recovery

CME's MDP 3.0 sends market data over **UDP multicast** with sequence numbers. Subscribers detect gaps and recover lost packets via a **separate TCP recovery channel**.

```
UDP MC stream: ... 100, 101, [lost], 103, 104, 105 ...
```

↑

subscriber sees gap

↓

```
TCP recovery query: "send me seq 102"
```

```
TCP recovery reply: <seq 102 payload>
```

The result

Hot path stays on UDP — **no HOL blocking**.

Recovery happens out-of-band on TCP — used **only** when a gap is detected.

Subscribers without gaps pay nothing for reliability they don't need.

Best of both worlds — UDP speed, TCP recovery, no head-of-line blocking.

Source: CME Group, "MDP 3.0 Channel Recovery", official market data documentation.

Top HFT Firms — **same pattern**, applied harder

Citadel Securities

UDP multicast feeds. FPGA-accelerated decode. ~25% of US retail equity volume.

Jane Street

Public engineering blog: "**P99 latency is what we optimize, not mean.**" OCaml + kernel bypass.

Jump Trading

Microwave network NY ↔ Chicago. Custom UDP-based protocols.

Virtu Financial

Market making at exchange co-location. UDP for tick data, TCP for orders, RDMA internally.

Same recipe at every firm: UDP for market data, TCP for orders, kernel bypass for both. Jane Street's emphasis on P99 over mean is *exactly* what our experiment in Part 3 measured.

Our Data ↔ Industry Practice

OUR EXPERIMENT SAID...	INDUSTRY DOES...
TCP P99 explodes to 205 ms under 1% loss	NASDAQ Market Data is UDP multicast (MoldUDP64)
UDP loses 8/1000 but the tail stays flat at ~374 μs	CME runs UDP + TCP recovery only on gaps (MDP 3.0)
TCP guarantees delivery at the cost of tail latency	NYSE Order Entry = TCP (correctness > speed for orders)
Baseline (no loss): TCP ≈ UDP, ~10 μs gap	Both layers further accelerated via kernel bypass (Citadel, Virtu)

We didn't just guess. We measured the exact trade-off the entire industry is built around.

Conclusion & Future Directions

What we learned

- ▶ HFT bottleneck is **software** — $d_{\text{proc}} + d_{\text{queue}}$, not physics.
- ▶ **Kernel bypass** (DPDK / OpenOnload / RDMA) attacks d_{proc} .
- ▶ **Protocol split** (UDP + TCP) attacks d_{queue} and tail latency.
- ▶ Both layers must be optimized together — neither alone is enough.

Future directions

- ▶ **QUIC / SRT** — user-space transports with built-in loss recovery.
- ▶ **SmartNICs / P4-programmable switches** — pushing logic into the network fabric.
- ▶ **FPGA acceleration** — tick-to-trade in hardware, at the cost of agility.
- ▶ The next 5 years: how much of a trading strategy can fit inside the NIC itself?

1 μ s is not just a number. It's the difference between a profitable trade and a missed one — and the industry has built an entire technology stack to chase it.

Q & A

Thank you. Offline Q&A available after the session.

Key References

- Aquilina, Budish & O'Neill (2022). *Quantifying the High-Frequency Trading "Arms Race"*. QJE.
- Lockwood et al. (2012). *A Low-Latency Library in FPGA Hardware for High-Frequency Trading*.
- Intel *DPDK Programmer's Guide*.
- CME Group — *MDP 3.0 Market Data Documentation*.
- NASDAQ — *TotalView-ITCH 5.0 Specification*.
- NYSE — *Pillar Gateway Technical Reference*.
- Kurose & Ross, *Computer Networking — A Top-Down Approach*, Chapters 1, 3.

Project Repo · github.com/Group07/Network_2026_1